

Sites LabMiM / LEAL

Onboarding da plataforma estática e do pipeline de dados

Objetivo

Uma aplicação compartilhada para publicações meteorológicas, conectada por contratos versionados a um produtor Python independente.

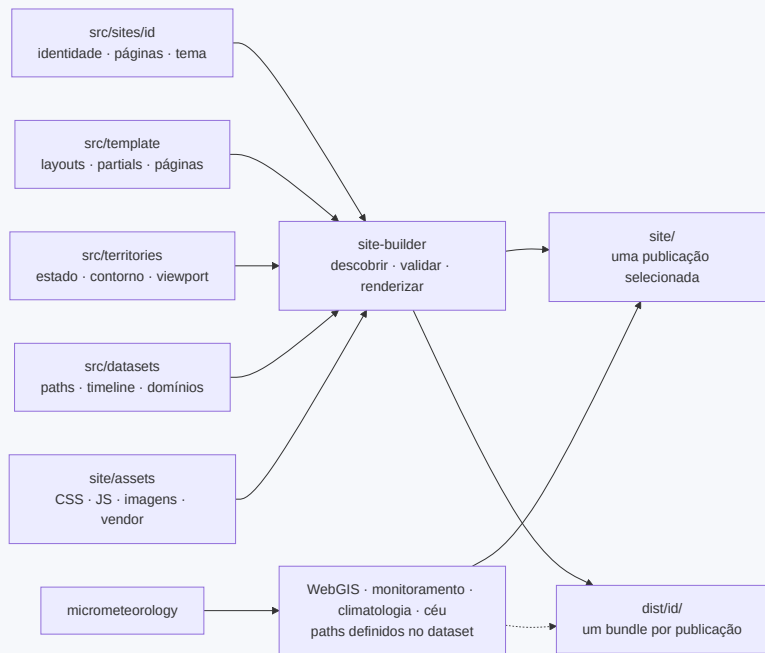
Hoje

- LabMiM / UFBA - Bahia
- LEAL / UFES - Espírito Santo
- Consumidor: `site-labmim`
- Produtor:
`micrometeorology/src/micrometeorology`

Camada	Escolha
Build	Node 24 + gerador próprio
Produtor	Python 3.14 + CLIs Typer
Saída	HTML, CSS e JS puros
Mapa	Leaflet 1.9.4
Gráficos	Chart.js 3.9.1
Dados	JSON, GeoJSON, binários e imagens

Node e Python ficam fora do servidor web. A publicação entregue continua inteiramente estática.

Mapa do repositório



Fontes editáveis

`src/template/` · `src/sites/` · `src/territories/` ·
`src/datasets/`

Saídas geradas

`site/` - uma publicação · `dist/<id>/` - todas

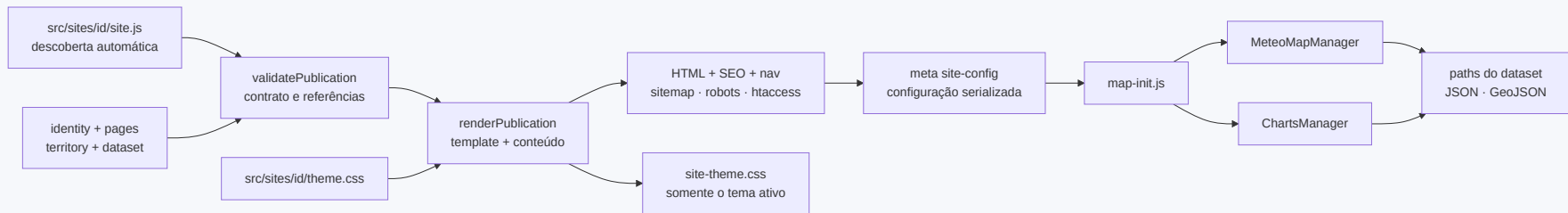
O modelo mental: quatro módulos

Módulo	Pergunta que responde	Local
Publicação	Quem publica? Qual conteúdo, SEO, navegação e marca?	<code>src/sites/<id>/</code>
Template	O que funciona igual em todos os sites?	<code>src/template/</code>
Território	Qual estado, contorno e enquadramento do mapa?	<code>src/territories/</code>
Dataset	Onde estão os dados, quais domínios e timeline?	<code>src/datasets/</code>

Um território e um dataset podem ser reutilizados por mais de uma publicação.

Evite `if (site === "ufba")` no template, no renderer, no CSS e no runtime.

Como o build compõe uma publicação



1 · Descobrir

`src/sites/*/site.js`

2 · Validar

schema, fontes, assets, GeoJSON,
páginas e redirects

3 · Renderizar

HTML, tema, SEO, sitemap, robots e
`.htaccess`

Descoberta: `site.js` é a composição

```
// src/sites/ufes/site.js
"use strict";

const identity = require("../identity");
const pages = require("../pages");
const dataset = require("../../datasets/leal-wrf");
const territory = require("../../territories/es");

module.exports = { ...identity, territory, dataset, pages };
```

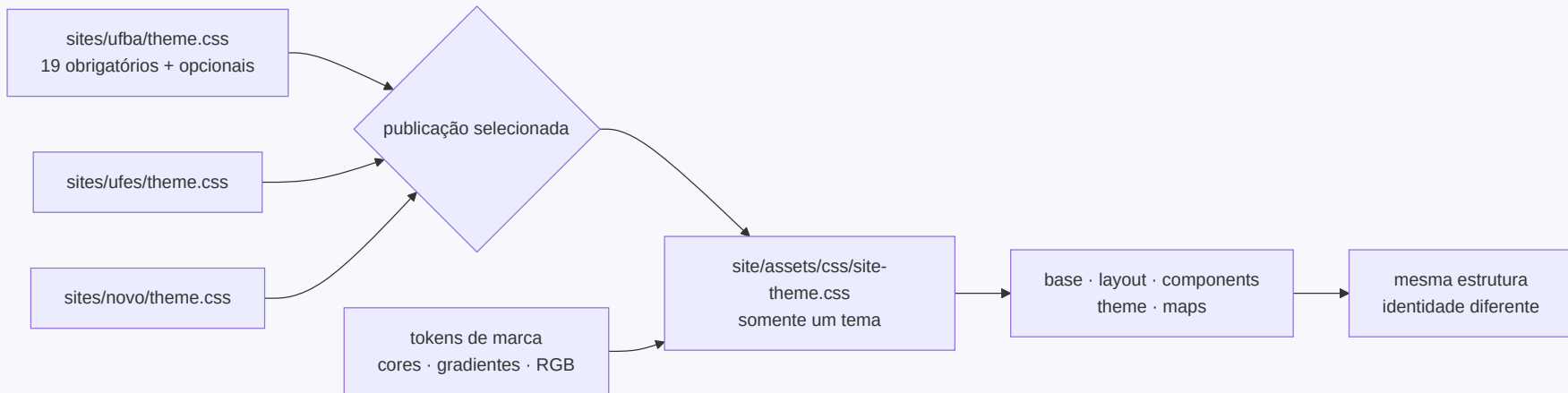
Registro automático

- uma pasta válida é o registro
- `sites:list` confirma descoberta
- `build:all` e `build:check` usam a mesma lista

Invariantes

- ID igual à pasta
- `origin` única, sem `/` final
- exatamente um `isDefault: true`
- `schemaVersion: 1`

CSS desacoplado por responsabilidade



Identidade

`src/sites/<id>/theme.css` - 19 tokens obrigatórios + opcionais, nenhum seletor.

Estrutura comum

`base · layout · components · theme`

Por página

`vendorStyles` + `styles`; WebGIS declara Leaflet e `maps.css`.

Onde colocar uma mudança de estilo?

Sempre carregado

- marca -> `src/sites/<id>/theme.css`
- reset/utilitário -> `base.css`
- navbar/footer -> `layout.css`
- cards/blocos -> `components.css`
- dark mode -> `theme.css`

Declarado por página

- WebGIS -> `maps.css` em `styles`
- asset comum -> `"assets/css/..."`
- fonte compartilhada -> `templateSource("styles/...")`
- fonte exclusiva -> `siteSource("styles/...")`
- biblioteca local -> `vendorStyles`

Não duplique estrutura num tema e não adicione seletor por ID no CSS comum.

Comandos e saídas

```
npm run sites:list  
npm run build           # publicação padrão -> site/  
npm run build -- --site=ufes # publicação escolhida -> site/  
npm run build:all       # todas -> dist/<id>/  
npm run build:check     # valida todas; restaura a padrão  
npm run lint:themes     # contrato e isolamento dos temas
```

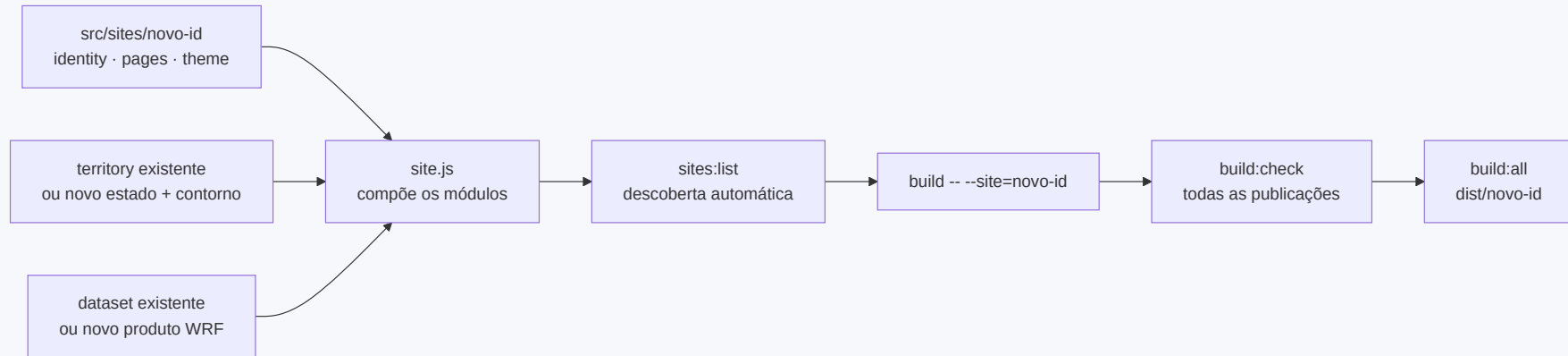
site/

- uma publicação por vez
- fluxo compatível com deploy atual
- preserva os paths operacionais do dataset
- remove HTMLs órfãos de outro site

dist/<id>/

- um frontend por publicação
- pronto para associação aos dados corretos
- omite os paths operacionais configurados
- `dist/` é git-ignored

Criar uma publicação nova: fluxo completo



Criar

```

src/sites/exemplo/
├─ site.js
├─ identity.js
├─ pages.js
├─ theme.css
├─ assets/      # imagens e logos próprios, opcional
├─ pages/
├─ styles/      # CSS estrutural exclusivo, opcional
└─ fragments/   # opcional
  
```

Não editar

- build.js
- package.json
- outputs derivados do manifesto
- template para trocar nomes de instituição
- CSS comum para trocar paleta

Passo 1: identidade

```
// src/sites/exemplo/identity.js
module.exports = {
  schemaVersion: 1,
  id: "exemplo",
  isDefault: false,
  origin: "https://exemplo.edu.br",
  brand: {
    name: "LAB",
    fullName: "Laboratório Exemplo",
    copyrightName: "LAB",
    ogImage: "assets/img/logo-exemplo.png",
    logos: { nav: { /* src + dimensões */ }, footer: { /* ... */ }, sidebar: { /* ... */ } },
    affiliations: [],
  },
  institution: { name: "Universidade Exemplo", acronym: "UE" },
  location: { cityName: "Cidade" },
  theme: "theme.css",
  redirects: [],
};
```

Assets nascem em `src/sites/<id>/assets/` e são publicados em `site/assets/`; redirects só podem chegar a páginas declaradas no próprio site.

Passo 2: território e dataset

src/territories/pe.js

```
module.exports = {
  id: "pe",
  kind: "state",
  code: "PE",
  name: "Pernambuco",
  regionPhrase: "região de Pernambuco e entorno",
  terrainExample: "o Planalto da Borborema",
  boundaryAsset: "assets/data/br_pe.json",
  viewport: {
    center: [-8.3, -37.8],
    zoom: 6,
    fitBoundary: true,
    fitMaxZoom: 7,
  },
};
```

src/datasets/exemplo-wrf.js

```
module.exports = {
  id: "exemplo-wrf",
  attribution: "LAB-UE",
  paths: {
    manifest: "JSON/manifest.json",
    values: "JSON",
    grids: "GeoJSON",
    monitoring: "Monitoramento", // opcional
    climatology: "Climatologia", // opcional
    sky: "Ceu", // opcional
  },
  timeline: {
    defaultMaxLayer: 75,
    initialIndex: 7,
    stepHours: 1,
    label: "Horário local (UTC-03)",
  },
  defaultDomain: "D01",
  domains: [/* id, label, centro, zoom, resolução */],
};
```

Montar o site

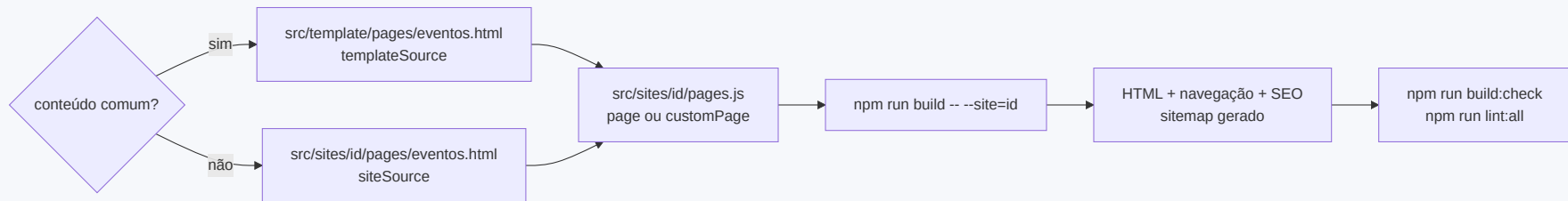
```
// src/sites/exemplo/pages.js
const { page, siteSource } = require("../../template/page-types");

module.exports = [
  page("home", {
    source: siteSource("pages/index.html"),
    seo: {
      h1: "LAB - Laboratório Exemplo",
      title: "LAB - Laboratório Exemplo · UE",
      description: "Pesquisa meteorológica da Universidade Exemplo.",
    },
  }),
  page("forecast", {
    seo: { title: "LAB - Previsões WRF · UE", description: "Previsões para Pernambuco." },
  }),
];
```

Implemente os 19 tokens obrigatórios e só os opcionais necessários em `theme.css`.

Componha tudo no pequeno `site.js`.

Adicionar uma página: primeiro decida o alcance



Compartilhada

Uma fonte em `src/template/pages/` ; cada publicação opta por incluí-la.

Exclusiva

Fonte em `src/sites/<id>/pages/` ; só aquele manifesto referencia.

Receita: página compartilhada

```
// em src/sites/<id>/pages.js
const { customPage, templateSource } = require("../../template/page-types");

customPage({
  id: "about-project",
  file: "projeto.html",
  layout: "institutional",
  source: templateSource("pages/projeto.html"),
  seo: {
    h1: "Sobre o projeto",
    title: "Sobre o projeto · LAB",
    description: "Objetivos, metodologia e resultados.",
  },
  nav: {
    label: "Projeto",
    icon: "fa-circle-info",
    order: 60,
    elementId: "nav-projeto",
  },
  styles: [templateSource("styles/projeto.css")],
});
```

Crie `src/template/pages/projeto.html` sem `<html>`, `<head>`, navbar ou footer.

Receita: página exclusiva ou variação editorial

```
const { customPage, siteSource } = require("../..//template/page-types");

customPage({
  id: "projeto-local",
  file: "projeto-local.html",
  layout: "institutional",
  source: siteSource("pages/projeto-local.html"),
  styles: [siteSource("styles/projeto-local.css")],
  seo: {
    h1: "Projeto local",
    title: "Projeto local · LEAL/UFES",
    description: "Iniciativa exclusiva no Espírito Santo.",
  },
  nav: false,
});
```

Anexar um trecho próprio

append: [siteSource("fragments/funding.html")]

Personalizar um tipo comum

sobrescreva `source`, `nav`, `styles` ou campos de SEO

Atualizar uma página existente: mapa de impacto

Conteúdo e catálogo

- comum -> `src/template/pages/*.html`
- exclusivo -> `src/sites/<id>/pages/*.html`
- anexo -> `src/sites/<id>/fragments/*.html`
- SEO/H1 -> `src/sites/<id>/pages.js`
- menu/ordem -> `page.nav` em `pages.js`

Estrutura e identidade

- head/navbar/footer -> `src/template/partials/`
- shell de página -> `src/template/layouts/`
- CSS compartilhado -> `site/assets/css/`
- CSS autoral -> `templateSource / siteSource`
- paleta -> `src/sites/<id>/theme.css`

Antes de editar algo compartilhado, procure os consumidores com `rg` e valide todas as publicações.

pages.js : uma fonte de verdade por site

Cada entrada governa:

arquivo · layout · fonte · fragmentos · styles/vendorStyles · SEO · navegação · indexação

Derivado automaticamente

- HTML completo
- item ativo da navbar
- links do rodapé
- canonical, Open Graph e Twitter
- sitemap e robots
- JSON-LD da home

Regras validadas

- IDs e outputs únicos
- uma home em `index.html`
- SEO completo
- ordem/ID/label de nav únicos
- fonte e layout existentes
- stylesheet seguro e existente

Território e dataset chegam ao runtime

```
territory + dataset  
-> renderer.runtimeConfig()  
-> <meta name="site-config" content="...">  
-> map-init.js / MeteoMapManager
```

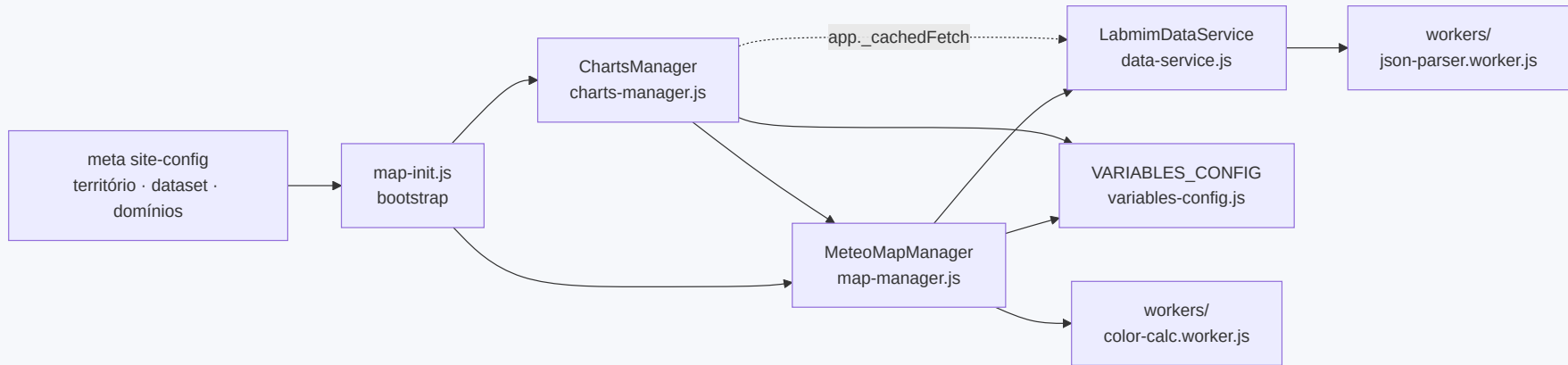
Território controla

- estado e sigla
- contorno local
- centro/zoom inicial
- `fitBounds` e zoom máximo
- textos geográficos do template

Dataset controla

- manifest, values e grids
- timeline fallback
- domínio padrão
- labels, centros e zooms
- documentação e atribuição

WebGIS compartilhado



`forecast` e `energy` reutilizam o layout WebGIS; contexto, conteúdo, estilos e SEO vêm da declaração da página.

Quatro famílias de dados chegam ao frontend

WebGIS WRF

- manifest v2 + JSON/GeoJSON
- `series.bin` via HTTP Range
- `summary.json` para prévia
- overlays `WIND_VECTORS` e `ISOBARS`

Monitoramento

- variante interativa `labmim-monitoring-v1`
- bruto + média horária + WRF
- variante estática: 9 PNGs fixos

Climatologia

- `labmim-climatology-v1`
- histogramas, ajustes e rosa dos ventos
- bibliografia e cobertura do registro

Condição do céu

- imagem all-sky + máscara
- Kt x Kd + modelos empíricos
- acumulada do índice de claridade

Todos chegam no deploy pelos caminhos de `dataset.paths`; `dist/<id>` não leva dados operacionais.

O que `build:check` protege

- **Registro** - ID/pasta, origem, padrão
- **Identidade** - ownership de assets, URLs, redirects
- **Tema** - tokens, seletores, pares hex/RGB
- **Território** - GeoJSON, sigla, viewport
- **Dataset** - paths, páginas de dados, timeline, domínios
- **Páginas** - fonte, layout, style, vendor, SEO/nav
- **Saída** - tokens, HTML, referências locais
- **CSS/vendor** - Bootstrap purgado e ícones
- **Bundles** - cada `dist/<id>` inclui só assets referenciados

Complementar: `npm run lint:all` · `npm run format:check` · inspeção manual em light/dark e mobile.

Riscos de regressão

- editar `site/*.html` -> mudança perdida
- identidade no template/CSS -> marca vaza
- copiar página comum -> conteúdo diverge
- `siteSource()` para capacidade comum -> duplicação
- esquecer manifesto consumidor -> catálogo desigual
- CSS sem `page.styles` -> estilo ausente
- testar CSS comum em um site -> regressão temática
- hardcode de estado/path -> mapa ou dados errados

Prevenção: fronteiras declarativas + `build:check` + inspeção das publicações afetadas.

Checklists operacionais

Nova publicação

- ☐ `identity.js`, `pages.js`, `theme.css`
- ☐ assets próprios no módulo da publicação
- ☐ páginas/fragments exclusivos
- ☐ território + contorno, se novo
- ☐ dataset, se novo
- ☐ composição em `site.js`
- ☐ `sites:list` e build individual
- ☐ `build:check` e `build:all`
- ☐ dados operacionais corretos no deploy

Página ou atualização

- ☐ decidir comum versus exclusiva
- ☐ editar fonte, não output
- ☐ atualizar SEO/nav em `pages.js`
- ☐ declarar `styles`, se houver
- ☐ verificar consumidores compartilhados
- ☐ build da publicação afetada
- ☐ `build:check` + `lint:all`
- ☐ light/dark, mobile e links
- ☐ contrato produtor/consumidor, se afetado

Dois repositórios, um contrato de publicação

O `site-labmim` **não produz dados meteorológicos**. Ele monta HTML e associa, no deploy, os artefatos gerados por um pipeline Python separado:

- `micrometeorology` - lê a saída do modelo **WRF** e a estação, e exporta os artefatos que as páginas do site consomem.

Os dois repositórios se ligam por **contratos versionados**: manifest, grade, valores, overlays, séries, monitoramento, climatologia e céu.

Uma mudança de formato exige PRs coordenados. O consumidor compatível deve chegar antes do produtor novo.

As 12 CLIs `labmim-*`

Publicação no site

`labmim-wrf-geojson` · `labmim-monitoring` · `labmim-climatology` · `labmim-sky` · `labmim-site-graphs`

Base operacional

`labmim-wrf-series` · `labmim-archive` · `labmim-sensor-process`

Figuras e análise

`labmim-wrf-figures` · `labmim-station-graphs` · `labmim-metrics` · `labmim-comparison`

`labmim-wrf-geojson` alimenta o WebGIS; as outras rotas do site possuem produtores próprios.

Organização do pipeline produtor

wrf/

Leitura NetCDF, variáveis, `value_source`, jobs, grades, séries, isóbaras e segurança de escrita.

sensors/

Ingestão, controle de qualidade, calibração, agregação, vento, arquivo e catálogo do monitoramento.

stats/

Climatologia, distribuições, radiação, Kt x Kd, comparação e métricas.

cli/ + common/

Entrypoints, configuração, logging, paths, tipos compartilhados e política de tempo local.

Entradas

`wrfout_d0X_*` · `.dat` /Parquet da estação · câmera all-sky

Processamento

Python 3.14 · NumPy/xarray/pandas · processos paralelos · escrita atômica

Saídas do site

WebGIS · monitoramento · climatologia · céu · PNGs

Classifique o produto antes de expandir o WebGIS

Campo cru

Já existe como escalar 2-D no `wrfout`.

`DEFAULT_VARS` + entrada no site.

Campo derivado

Exige fórmula, conversão ou vários campos.

`variables.py` + `value_source.py` +
tipo/id + defaults.

Overlay

Geometria desenhada sobre um campo.

Work unit próprio + descriptor em
`manifest.features`.

produtor -> `{D}_{ID}_{NNN}.json` + `manifest.json` -> consumidor

Campos sombreados entram em `VARIABLES_CONFIG` e em
`VARIABLE_CONTEXTS`.

Overlays como `ISOBARS` ficam fora do catálogo de base e
são descobertos em `features`.

Receita A - campo escalar cru (2 edições)

Pipeline · cli/export_wrf_geojson.py

```
DEFAULT_VARS = [
    "temperature", "wind", "rain",
    # exemplo: campo escalar que o wrfout já carrega
    "UST",          # velocidade de fricção
]
# 0 ramo genérico em value_source.py já cobre:
# ds.has_variable("UST") -> extract_scalar
# id de saída = "UST".upper()
```

```
$ labmim-wrf-geojson -v UST \
    -o site/JSON -g site/GeoJSON -D 1,4
# -> D01_UST_007.json, D01_UST.series.bin
```

Site · site/assets/js/variables-config.js

```
frictionVelocity: {
  id: "UST", sourceId: "UST", // == {VAR}
  label: "Velocidade de fricção", unit: "m/s",
  faIcon: "wind",
  colors: ["#f7fbff", /* rampa editorial */ "#08306b"],
  scaleMin: 0, scaleMax: 2, // validar com dados reais
  summary: "Escala turbulenta próxima à superfície.",
  specificInfo: (v) => v == null
    ? unavailableInfo("Velocidade de fricção") : { /* ... */
},
// e expor a chave "frictionVelocity" no array
// VARIABLE_CONTEXTS.forecast.variables
```

Receita B - variável derivada em quatro camadas

1 • Extractor · wrf/variables.py

```
def extract_relative_humidity(ds):
    q2 = ds.get_variable("Q2") # kg/kg
    t2 = ds.get_variable("T2") # K
    psfc = ds.get_variable("PSFC") # Pa
    rh = compute_relative_humidity(q2, t2, psfc)
    lo, hi = percentile_scale_bounds(rh)
    return rh, lo, hi # (vals3d, vmin, vmax)
```

2 • Tipo + id · common/types.py

```
class WRFVariable(StrEnum):
    RELATIVE_HUMIDITY = "relative_humidity"

VARIABLE_NETCDF_MAP = {
    WRFVariable.RELATIVE_HUMIDITY: "RH2", # -> {VAR}
}
```

3 • Dispatch compartilhado · wrf/value_source.py

```
elif variable_name == WRFVariable.RELATIVE_HUMIDITY:
    values, lo, hi = \
        variables.extract_relative_humidity(dataset)

    return ValueFrameSource(
        frame_for_step=lambda i: variables.materialize_2d(
            values[i : i + 1]
        ),
        scale_min=lo, scale_max=hi,
    )
```

4 • Habilitar · cli/export_wrf_geojson.py

```
DEFAULT_VARS = [ ..., "relative_humidity" ]
```

Depois, no site: entrada em `VARIABLES_CONFIG` com `sourceId: "RH2"` (igual ao `{VAR}` do arquivo).

IDs, overlays e escolhas editoriais

O contrato de id

O `{VAR}` de `{D}_{VAR}_{NNN}.json` vem de `VARIABLE_NETCDF_MAP[nome]` (ou `nome.upper()`).

O `sourceId` no site **tem** de ser esse `{VAR}`. Confira em `JSON/manifest.json`.

Base x overlay

`WIND_VECTORS` e `ISOBARS` são descobertos em `manifest.features`; não entram em `VARIABLES_CONFIG` nem geram série de célula.

`sourceId` errado -> 404 + cache negativo; a base fica muda sem erro visual explícito.

Escala, paleta e ícone

`colors[]` é uma rampa; a cor por célula interpola entre `scaleMin` e `scaleMax` fixos no site.

Isso é **decisão editorial**. Ícone novo exige regenerar o subset Font Awesome.

Fontes: `wrf/variables.py` · `wrf/value_source.py` · `wrf/jobs.py` · `wrf/isobars.py` · `common/types.py` · `site/assets/js/variables-config.js`

Referências rápidas

Comandos - site

```
npm run sites:list  
npm run build -- --site=<id>  
npm run build:all  
npm run build:check  
npm run lint:all
```

Comandos - pipeline de dados

```
labmim-wrf-geojson -v <var> \  
  -o site/JSON -g site/GeoJSON -D 1,4  
labmim-monitoring -i <archive> -o site/Monitoramento \  
  -w data/series_operacional.dat
```

Guias: [src/sites/README.md](#) ·

[micrometeorology/docs/micrometeorology.md](#)

Documentação técnica

- [Architecture.md](#)
- [README.md](#)
- [docs/onboarding-architecture/architecture-evidence.md](#)

Código do gerador

- [scripts/site-builder/{publications,validate,renderer}.js](#)
- [src/template/page-types.js](#)

Pipeline de dados (micrometeorology)

- [wrf/variables.py](#) · [wrf/value_source.py](#) · [wrf/jobs.py](#)
- [wrf/isobars.py](#) · [wrf/operational_series.py](#)